

Step-by-Step: Building the Seller Acquisition ROI Model

Six weeks at 1–2h/day, on **Olist's Marketing Funnel** (acquisition) + **Olist's Brazilian E-Commerce** (outcome), joined on the one key that's real, `seller_id`, and honest about how rarely it resolves.

Realistic scheduling: finish weeks 1–3 before you start the job. Do 4–6 after, at whatever pace the job leaves you.

Ground rules

- Git from commit #1, one branch per week, merged via a PR to yourself.
 - Never commit a service-account key. `gcloud auth application-default login` locally, `*.json` and `.env` in `.gitignore`.
 - Never commit the data. Commit `download.py`. Both datasets are CC-BY-NC-SA-4.0 — non-commercial, share-alike; redistribution of the raw files isn't yours to grant.
 - **Develop against DuckDB, ship against BigQuery.** Same models, two targets. See Appendix C.
 - Budget alert at €5 before your first cloud query.
 - Every week ends with something that runs.
-

Day 0 — Setup (~90 min)

What	Link
Google Cloud account + credits	https://cloud.google.com/free
gcloud CLI	https://cloud.google.com/sdk/docs/install
Docker Desktop (give it ≥4 GB RAM)	https://docs.docker.com/get-started/
uv for Python environments	https://docs.astral.sh/uv/getting-started/
DuckDB CLI + Python	https://duckdb.org/docs/installation/
Kaggle API	https://github.com/Kaggle/kaggle-api

Create project `pn1-forecast` (GCP will suffix it with a random number if the short name's taken — that's normal, just use whatever it gives you consistently from here on), enable BigQuery and Cloud Storage APIs, set the budget alert. If you already have a bucket from an earlier attempt at this project, keep its name even if it doesn't match the project's current name — GCS bucket names can't be changed in place, and renaming means create-copy-delete, not worth it for a dev bucket.

Done when: `bq ls` runs, `duckdb -c "select 1"` runs, and `kaggle datasets list -s olist` returns something.

Week 1 — Ingestion and Python hygiene

Learn

- uv project layout — <https://docs.astral.sh/uv/guides/projects/>
- pytest — <https://docs.pytest.org/en/stable/getting-started.html>
- Docker concepts — <https://docs.docker.com/get-started/introduction/>
- Git branching (Pro Git ch. 3) — <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
- Parquet, and why you convert immediately — <https://parquet.apache.org/docs/>

Build

`download.py` fetching both sources:

```
kaggle datasets download -d olistbr/brazilian-ecommerce      # ~126 MB, 9
tables
kaggle datasets download -d olistbr/marketing-funnel-olist  # ~0.9 MB, 2
tables
```

The `?select=` param in a Kaggle dataset URL is a UI deep-link only. `.../brazilian-ecommerce?select=olist_orders_dataset.csv` just opens the preview pane on that one file — `kaggle datasets download` always pulls the whole zip regardless of what the URL points the browser at. There's no per-file download in the API.

Neither dataset is a competition, so there's no rules-acceptance click-through to worry about — unlike a Kaggle *competition*, `kaggle datasets download` on a public dataset works the moment your API key is in place.

Then `ingest.py`: CSV → Parquet → `gs://<bucket>/raw/olist_ecom/<table>/` and `.../raw/olist_funnel/<table>/`. Partition `orders` by month on `order_purchase_timestamp`. Use `logging`, not `print`. One pytest test on the path builder.

Do the conversion with DuckDB, not pandas, even though nothing here is 125M rows this time — `geolocation` is the one table with real heft (1,000,163 rows), and the habit of streaming through DuckDB's `COPY` rather than loading a full CSV into a pandas `DataFrame` is the thing worth building now, before you're on a table that would actually punish you for skipping it. See `business-case.md` §6 for why this project treats the working volume as standing in for a multi-GB production system even though the Kaggle export itself is modest.

```
COPY (SELECT * FROM read_csv_auto('olist_geolocation_dataset.csv')) TO
'out/' (FORMAT PARQUET)
```

Done when: running twice leaves the bucket unchanged, and it runs in a container.

Be able to explain: why raw keeps source shape and does no cleaning, and why columnar Parquet beats CSV here even at this modest a scale.

Week 2 — Two engines, one dataset

Learn

- Loading Parquet from GCS — <https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet>
- Partitioned tables — <https://cloud.google.com/bigquery/docs/partitioned-tables>
- Clustered tables — <https://cloud.google.com/bigquery/docs/clustered-tables>
- Cost best practices — <https://cloud.google.com/bigquery/docs/best-practices-costs>
- External tables — <https://cloud.google.com/bigquery/docs/external-data-cloud-storage>
- DuckDB reading Parquet — <https://duckdb.org/docs/data/parquet/overview>
- DuckDB over httpfs/GCS — <https://duckdb.org/docs/extensions/httpfs/overview>

Build

1. `load.py` → `raw.olist_orders` partitioned by `order_purchase_timestamp`, clustered by `order_status`; the other eight tables unpartitioned (too small to matter, including `geolocation` — row count isn't the same axis as byte volume, and even 1M rows of five `FLOAT64` columns is a rounding error against the partition-worthiness bar).
2. The same tables queryable locally in DuckDB straight off the Parquet.
3. **The cost experiment.** Create an unpartitioned copy of `orders`. Run the same one-month query against both and record the bytes-scanned estimate. Then run the same query in DuckDB and record wall-clock time. Write all three numbers in your README. Yes, the absolute bytes will be tiny either way — the point is the *ratio* between partitioned and unpartitioned, which holds regardless of table size, and which is the actual thing you're meant to be learning to spot before it costs real money on real data.

Done when: you have that three-way comparison written down, and reloading a month replaces rather than duplicates.

Be able to explain: partition pruning; why `LIMIT` doesn't reduce BigQuery cost; why selecting fewer columns does.

Week 3 — dbt, portable across both engines

Learn

- Start here — <https://docs.getdbt.com/docs/get-started-dbt>
- dbt Fundamentals, free, ~5h — <https://learn.getdbt.com/catalog?category=beginner>
- dbt Core install — <https://docs.getdbt.com/docs/core/installation-overview>
- BigQuery setup — <https://docs.getdbt.com/guides/bigquery>
- dbt-duckdb adapter — <https://github.com/duckdb/dbt-duckdb>
- Project structure — <https://docs.getdbt.com/best-practices/how-we-structure/1-guide-overview>
- Cross-database macros — <https://docs.getdbt.com/reference/dbt-jinja-functions/cross-database-macros>
- Tests — <https://docs.getdbt.com/docs/build/data-tests> · Seeds — <https://docs.getdbt.com/docs/build/seeds>

Build

- One source group, `stgolist__*`, covering both Kaggle downloads — they share a real key (`seller_id`) this time, so there's no "never join" rule to enforce. There is a "join left, and check the match rate" rule instead.
- `intolist__funnel_weekly` — MQL count, closed-deal count, and conversion rate, by `origin` × ISO week.
- `intolist__seller_monthly_gmv` — GMV (`sum(price)`), order count, freight, average review score, by `seller_id` × calendar month. Join `order_items` → `orders` (for the purchase timestamp) → `order_reviews`.
- `intolist__seller_acquisition` — one row per seller that actually has a `closed_deals` record (380 of them): origin, lead_type, business_type, SDR/SR ids, days from `first_contact_date` to `won_date`. **Left join `seller_monthly_gmv` onto this, not the other way round** — the outcome table is the spine; the acquisition table is enrichment that's absent 88% of the time, and absence has to survive the join as `null/est_is_attributed = false`, not silently filter rows out.
- Seeds: `channel_cost_benchmarks.csv` (origin, cost-per-lead, SDR hours-per-close, source column) and `scenario_parameters.csv`.
- `fct_seller_ltv` and `fct_channel_funnel` applying the model from `business-case.md` §C. **Prefix every synthesised column `est_`.**
- Tests: `unique/not_null` on keys, `relationships` to dimensions, `accepted_range` on `est_is_attributed=true` rows' `days_to_close` (should be non-negative), and a singular test asserting the attributed-seller count in `fct_seller_ltv` never exceeds the 380 you counted by hand in Week 1 — that number should only ever go down if a dbt refactor breaks the join, never up.

Done when: `dbt build --target dev` (DuckDB) and `--target prod` (BigQuery) both pass from the same SQL.

Be able to explain: why the acquisition join is a left join off the outcome table and not the reverse, and what `est_is_attributed = false` on 88% of sellers actually means for any channel-level conclusion you draw later.

Week 4 — Orchestration and infrastructure

Learn

- Airflow tutorial — <https://airflow.apache.org/docs/apache-airflow/stable/tutorial/index.html>
- Airflow with Docker Compose — <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html>
- Terraform on GCP — <https://developer.hashicorp.com/terraform/tutorials/gcp-get-started>
- Google provider reference — <https://registry.terraform.io/providers/hashicorp/google/latest/docs>

Build

Weekly DAG: `ingest` → `load` → `dbt build`, with retries. Terraform for bucket and datasets, `terraform destroy` working. README with the architecture diagram.

Done when: you can destroy everything and rebuild in under 30 minutes from your own README.

Be able to explain: what happens when a task retries mid-write, and why every step is idempotent.

Week 5 — Forecasting

Learn

- statsmodels time series — <https://www.statsmodels.org/stable/tsa.html>
- Survival/decay curves — [https://lifelines.readthedocs.io/en/latest/Survival analysis with lifelines.html](https://lifelines.readthedocs.io/en/latest/Survival%20analysis%20with%20lifelines.html) (lifelines, optional but a natural fit for a retention curve)
- Rolling-origin backtesting — <https://otexts.com/fpp3/tscv.html> (Hyndman & Athanasopoulos, free)
- LightGBM — <https://lightgbm.readthedocs.io/en/stable/>

Build

- Cohort-naive baseline first.** A seller's GMV at month t since close = the attributed cohort's observed median GMV at month t . Measure WAPE. This is the bar.
- A retention/decay curve fit on the 380 attributed sellers' actual post-close trajectories, 13-week (one-quarter) horizon.
- Rolling-origin backtest, by cohort month rather than calendar month — a cohort's "month 3" isn't the same calendar month for a seller who closed in January vs. one who closed in October.
- `ltv_forecast.py` writes to `marts.fct_ltv_forecast` with a `model_version` column.

Done when: you can state how much better than baseline you are, and say out loud that 380 sellers over ~11 tracked months is a small n for fitting a decay curve — don't let the forecast's own confidence outrun what it was actually fit on.

Be able to explain: why WAPE rather than MAPE when many cohort-months have near-zero GMV (a seller's first month, or a seller who churned).

Guardrail: if the decay curve loses to cohort-naive, ship cohort-naive and say so.

Week 6 — Scenarios and sensitivity

Learn

- Jinja and variables — <https://docs.getdbt.com/docs/build/jinja-macros>
- SALib — <https://salib.readthedocs.io/en/latest/>
- Streamlit — <https://docs.streamlit.io/>

Build

- Populate `scenario_parameters.csv`: `scenario_name`, `close_rate_multiplier`, `take_rate`, `cac_multiplier`, `sdr_hours_multiplier`, `retention_half_life_months`, `discount_rate`. Rows for base / pessimistic / optimistic / sdr_capacity_crunch / cac_inflation / retention_shock.
- `fct_roi_scenario_grid`: cross join forecast × seed, apply the ROI formula. Pure SQL.
- Run it on BigQuery, then think.** 13 weeks × ~10 channels × 6 scenarios is trivial. Now imagine the same fan-out at seller × week grain instead of channel × week — 3,095 sellers × 13 weeks × 6 scenarios is two orders of magnitude more rows from the scenario fan-out alone. That contrast is the lesson, same as before.
- Tornado chart: OAT across each parameter's range, sorted by ROI swing.

5. Monte Carlo in `simulate.py`: triangular distributions, 10k draws, P10/P50/P90 into `fct_roi_dist`.
6. Add both to the DAG. One page of output.

Done when: you can name the top three ROI drivers by variance contribution and defend the ranking.

Be able to explain: why CAC being paid up front and revenue being a conditional, decaying stream makes the downside tail worse than a naive symmetric-uncertainty view would predict — a channel with a great close rate feeding sellers who never ship is a real failure mode this model has to be able to show.

Appendix A — Datasets

Brazilian E-Commerce (outcome / GMV backbone)

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

99,441 orders, Sep 2016 – Oct 2018. Nine tables: `orders` (status, purchase/delivery timestamps), `order_items` (112,650 rows — `order_id`, `seller_id`, `price`, `freight_value`), `order_payments`, `order_reviews`, `customers`, `sellers` (3,095 rows), `products`, `geolocation` (1,000,163 rows — zip-prefix centroid lat/lng), and `product_category_name_translation`.

Why it's the outcome source: it's where GMV, order volume, and seller lifespan actually live — the number the acquisition side is ultimately trying to justify spending against.

The structural quirk that matters most: `seller_id` is the only key this dataset shares with the funnel dataset, and it only resolves for 380 of the 3,095 sellers here (see `business-case.md` §3). Every other join in this project is entirely within this dataset.

Licence: CC-BY-NC-SA-4.0 — non-commercial, share-alike. Don't redistribute the raw files.

Marketing Funnel by Olist (acquisition source)

<https://www.kaggle.com/datasets/olistbr/marketing-funnel-olist>

Two tables. `marketing_qualified_leads` (8,000 rows, Jun 2017 – May 2018): `mql_id`, `first_contact_date`, `landing_page_id`, `origin` (organic_search, paid_search, social, direct_traffic, email, referral, display, other, unknown). `closed_deals` (842 rows, Dec 2017 – Nov 2018): adds `seller_id`, `sdr_id` (Sales Development Rep — qualifies the lead), `sr_id` (Sales Rep — closes it), `won_date`, `business_segment` (34 distinct values), `lead_type`, `business_type` (reseller / manufacturer), and some sparsely-populated declared fields (`declared_monthly_revenue`, `declared_product_catalog_size`) that are mostly null and worth checking before you rely on them for anything.

Used to derive: weekly lead volume and close rate by `origin`, and — via the `seller_id` join — which of the sellers actually generating GMV have a tracked acquisition channel at all.

Licence: CC-BY-NC-SA-4.0, same as above.

Exogenous ranges (for defensible sensitivity bounds)

- **SaaS/marketplace CAC and payback-period benchmarks** — used to anchor `channel_cost_benchmarks.csv`; cite whatever specific report you pull the range from in the seed's `source` column, not just "industry benchmark."
- **Marketplace commission (take rate) ranges** — Olist doesn't publish its actual rate; anchor `take_rate` on published ranges for comparable marketplaces and state the range explicitly.

These turn `cac_multiplier` and `take_rate` from invented numbers into cited ones.

Appendix B — The GPU evening (separate track)

Not on the critical path. Do it on a weekend.

- Stage 1 — run models on the GPU (~2h):** NVIDIA driver + CUDA <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/> · PyTorch <https://pytorch.org/get-started/locally/> · Ollama <https://ollama.com/>.
Success = `torch.cuda.is_available()` is True.
- Stage 2 — use it here (~1h, optional):** `XGBoost(device="cuda")`
<https://xgboost.readthedocs.io/en/stable/gpu/index.html> · cuDF <https://docs.rapids.ai/api/cudf/stable/> for a 10M-draw Monte Carlo.
- Stage 3 — actually learn CUDA (months, defer):** <https://developer.nvidia.com/blog/even-easier-introduction-cuda/> · <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> · Numba <https://numba.readthedocs.io/en/stable/cuda/index.html>

Appendix C — Hosting, queries, and which engine runs what

The most common confusion in this project. Read it once before week 2.

C.1 Four places data can live

Regime	Who pays storage	Who pays compute	Applies to you?
<code>bigquery-public-data</code>	Google	You, per byte scanned	No — neither dataset is here
Your BigQuery native tables	You	You, per byte scanned	Yes — main path
Your GCS + BigQuery external table	You (GCS only)	You, per byte scanned	Optional — avoids double storage
Your local disk + DuckDB	Nobody	Your own CPU/RAM	Yes — dev loop

The free public datasets you read about are regime 1: Google eats the storage bill, which is why you can query a 3 TB table for free. **Both Olist datasets are Kaggle downloads.** The moment you use them, you leave that regime and become the host. That changes the economics, though — as with the previous version of this project — not painfully, and this time not even close to painfully: see the scale note below.

C.2 What it actually costs, and the declared-scale caveat

Getting data in is free: GCS ingress isn't billed, and batch-loading from GCS into BigQuery carries no per-byte charge. You pay to *keep* and to *scan*.

- **Storage:** first 10 GiB free, then roughly \$0.02/GB/month for active storage, halving after 90 days untouched.
- **Queries:** first 1 TiB scanned per month free, then about \$6.25/TiB.

Actual sizing, verify against the console rather than trusting these:

	Raw CSV	As Parquet	In BigQuery
E-commerce (9 tables, incl. geolocation)	~126 MB	~30–50 MB	~40–60 MB
Funnel (2 tables)	~0.9 MB	~0.3 MB	~0.5 MB

Both fit inside the free 10 GiB with room to spare — by a much wider margin than Favorita's ~1–2 GB did in the previous version of this project. [business-case.md §6 states plainly why this project still practices partitioning and cost discipline as if the real number were multiple GB](#): the working assumption is that this public export is a bounded sample of a production system running at that scale, and the pipeline is built to the standard that assumption implies. The bytes-scanned numbers you'll actually see running these exercises are genuinely tiny — that's expected, not a sign you did something wrong.

The honest implication: at real Olist-export scale, you would need something like 20,000 careless full scans in a month to pay anything. Cost discipline here is a habit built by *practicing the pattern*, not by feeling real pain — so, deliberately, in week 2: build the unpartitioned twin, compare estimates, and run one query against [bigquery-public-data.wikipedia.pageviews_*](#) with and without a date filter to watch three orders of magnitude appear and disappear on a table where the stakes are real.

C.3 How BigQuery bills, precisely

Billing is on **bytes of the columns your query touches**, decompressed, across the partitions it can't prune. Three consequences that trip people up:

- **LIMIT 10** costs the same as no limit. The limit applies after the scan.
- **SELECT *** on a wide table is the single most expensive thing you can write. Naming five columns instead of forty cuts the bill roughly eightfold.
- A filter on the partition column prunes. A filter on any other column doesn't — clustering makes it *faster* and *cheaper* in practice, but it's a block-skipping optimisation, not a guarantee.

Check before you run, always:

```
bq query --dry_run --use_legacy_sql=false 'SELECT ...'
```

The console shows the same estimate in the top-right as you type. It's free to look, and it's the habit that separates people who get a surprise invoice from people who don't.

Enforce it in code too — `maximum_bytes_billed` makes an over-budget query fail rather than charge you, and it belongs in your dbt profile:

```
prod:
  type: bigquery
  maximum_bytes_billed: 5000000000 # 5 GB ceiling; the query errors
  instead of billing
```

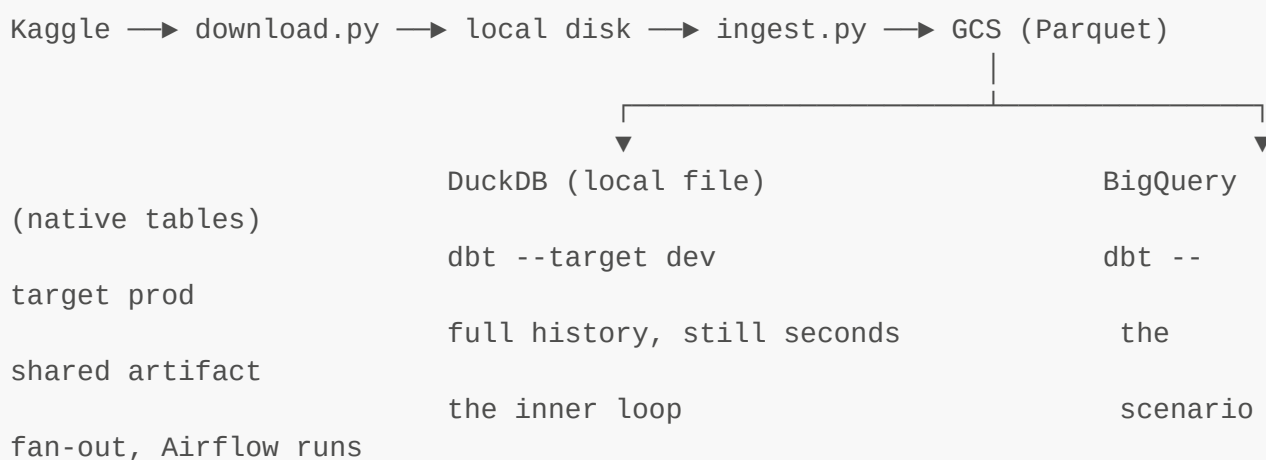
C.4 What DuckDB is doing differently

DuckDB is an in-process columnar engine — a library, not a server. It's vectorised, multi-threaded, and spills to disk when a working set exceeds RAM. It reads Parquet natively, including directly from GCS via the `httpfs` extension, so the same files back both engines.

On your laptop, every query in this project returns in well under a second against local Parquet — there's no table here large enough to make DuckDB sweat, which is itself worth noticing: the previous version of this project used a 125M-row table to make the pandas-vs-DuckDB gap undeniable, and that gap doesn't show up at this data's actual size. What still holds regardless of scale: zero cost, no network round trip, and no per-query floor. For an inner development loop where you run `dbt build` forty times an evening, that difference dominates everything else even on small data.

What it doesn't give you: concurrent access for other people or tools, managed governance, or elastic scale-out. Those are the actual reasons a client is on BigQuery, and none of them show up in a solo project — worth understanding, because it means you shouldn't conclude from this project that warehouses are pointless.

C.5 The division of labour



GCS is the single source of truth. The DuckDB file is disposable — rebuilt from GCS on whichever machine you're sitting at, which is exactly how you move between your desktop and your laptop without syncing anything.

Unlike the previous version of this project, there's no need for a `dev` date-range sample var here — the full history fits comfortably in the inner loop already. If you want to practice the sampling pattern anyway (you'll need it on a real client's actual multi-GB table), apply it to `geolocation`, the one table with real row count:

```
{% if target.name == 'dev' %}
  where geolocation_state in ('SP', 'RJ') -- practice sampling even though
you don't strictly need it here
{% endif %}
```

C.6 Keeping one set of SQL portable

Put both targets in `profiles.yml`:

```
pnl_forecast:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: 'data/local.duckdb'
      extensions: [httpfs, parquet]
    prod:
      type: bigquery
      method: oauth
      project: pnl-forecast
      dataset: analytics
      location: EU
      maximum_bytes_billed: 5000000000
```

Then keep the SQL neutral. dbt's cross-database macros exist for exactly this — `{{ dbt.date_trunc('week', 'order_purchase_timestamp') }}` compiles correctly on both. Reach for `{% if target.type == 'bigquery' %}` only where you genuinely can't avoid divergence.

Dialect differences you *will* hit:

	BigQuery	DuckDB
Types	INT64, FLOAT64, STRING	BIGINT, DOUBLE, VARCHAR
Safe cast	SAFE_CAST(x AS INT64)	TRY_CAST(x AS BIGINT)
Week truncation	DATE_TRUNC(d, WEEK(MONDAY))	date_trunc('week', d)
Filtered aggregate	not supported — use CASE WHEN	count(*) FILTER (WHERE ...)
Identifiers	backticks, project.dataset.table	double quotes, schema.table
Reading Parquet	external table or load job	read_parquet('gs://...') inline

Two rules keep this manageable: put all engine-specific weirdness in the staging layer so intermediate and marts stay clean, and run `dbt build --target prod` at least once a week. Portability that isn't tested

isn't portability.

C.7 The external-table alternative

If you'd rather not store the data twice, define a BigQuery external table over the Parquet already in GCS. You then pay GCS storage only, and BigQuery reads through to the files.

Worth doing once so you understand it. The trade-offs are real: no clustering, no native partition metadata (you get pruning only if the files are laid out `dt=2017-01-01/...` and you declare hive partitioning), and consistently slower scans. Native storage is the better default here — the data is small enough that the storage saving is pennies, and the query experience is meaningfully better.

Tool versions and doc URLs drift. If a link 404s, go to the tool's docs root rather than trusting an old blog post. Airflow changed substantially between 2.x and 3.x — always check which version a tutorial targets.